

توضیحات کد:

(1) وارد کردن کتابخانه‌ها

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
import scipy.io.wavfile as wavfile
```

توضیح

در این قسمت سه کتابخانه موردنیاز وارد برنامه می‌شوند.

- **NumPy** برای انجام محاسبات عددی و پردازش آرایه‌ها استفاده می‌شود.
- **Matplotlib** برای رسم نمودارهای سیگنال در حوزه زمان و فرکانس به کار می‌رود.
- **scipy.io.wavfile** برای خواندن و ذخیره فایل‌های صوتی با فرمت WAV استفاده می‌شود.

(2) خواندن فایل صوتی

```
fs, data = wavfile.read('original.wav')
```

توضیح

این دستور فایل صوتی را می‌خواند.

دو خروجی تولید می‌شود:

- **fs** : نرخ نمونه‌برداری (Sampling Frequency)

- **data** : نمونه‌های صوتی

مثلاً اگر

$fs = 44100$

باشد یعنی در هر ثانیه ۴۴۱۰۰ نمونه از صدا ثبت شده است.

(3) ساخت محور زمان

```
t = np.linspace(0, len(data)/fs, len(data))
```

توضیح

برای رسم سیگنال در حوزه زمان باید محور زمان ساخته شود.

در این دستور:

- زمان از صفر شروع می‌شود.
- تا مدت کل فایل صوتی ادامه پیدا می‌کند.
- تعداد نقاط زمان برابر تعداد نمونه‌های صوتی است.

(4) رسم سیگنال اولیه

```
plt.figure(1)
```

```
plt.plot(t, data)
```

```
plt.xlabel('Time')
```

```
plt.ylabel('Amplitude')
```

```
plt.title('Time Domain')
```

توضیح

این قسمت شکل موج اصلی صدا را رسم می‌کند.

محور افقی:

زمان (ثانیه)

محور عمودی:

دامنه سیگنال صوتی (Amplitude)

(5) انجام تبدیل فوریه سریع (FFT)

```
dataf = np.fft.fft(data)
```

توضیح

در این مرحله سیگنال از حوزه زمان وارد حوزه فرکانس می‌شود. FFT مشخص می‌کند که صدا از چه فرکانس‌هایی تشکیل شده است و شدت هر فرکانس چقدر است.

(6) محاسبه دامنه فرکانس‌ها

```
fd = abs(dataf)
```

توضیح

خروجی FFT اعداد مختلط است. برای مشاهده شدت هر فرکانس فقط اندازه (Magnitude) آن نیاز است.

به همین دلیل از تابع

`abs()`

استفاده شده است.

(7) انتخاب نیمه طیف

```
x = fd[0:int(len(fd)/2)-1]
```

توضیح

طیف FFT متقارن است. بنابراین فقط نیمه اول طیف کافی است و نیمه دوم اطلاعات تکراری دارد.

8) ساخت محور فرکانس

```
faxis = np.linspace(0, fs/2, len(x))
```

توضیح

محور افقی نمودار فرکانس ساخته می‌شود.

طبق قضیه نایکوئیست، بیشترین فرکانس قابل نمایش برابر است با:

$$\left[\frac{fs}{2} \right]$$

مثلاً اگر

$$fs = 44100$$

باشد،

حداکثر فرکانس برابر است با

$$22050 \text{ Hz}$$

9) رسم طیف فرکانسی

```
plt.figure(2)
```

```
plt.xlabel('Frequency')
```

```
plt.ylabel('Amplitude in dB')
```

```
plt.title('Frequency Domain')
```

```
plt.xscale('log')
```

```
plt.plot(faxis, 20*np.log10(x/len(data)))
```

توضیح

در این قسمت طیف فرکانسی رسم می‌شود.

- محور افقی = فرکانس
- محور عمودی = شدت سیگنال بر حسب دسی‌بل

همچنین

`plt.xscale('log')`

باعث می‌شود محور فرکانس لگاریتمی شود تا فرکانس‌های پایین بهتر دیده شوند.

(10 تعیین محدوده فرکانس‌ها)

```
k1 = int(len(dataf)/fs * 1000)
```

```
k2 = int(len(dataf)/fs * 10000)
```

توضیح

اینجا مشخص می‌شود که فرکانس‌های

1000 Hz

تا

10000 Hz

در آرایه FFT در چه اندیس‌هایی قرار دارند.

بعداً این قسمت تقویت خواهد شد.

11) تعیین محدوده نویز

```
n1 = int(len(dataf)/fs * 0.1)
```

```
n2 = int(len(dataf)/fs * 90)
```

```
n3 = int(len(dataf)/fs * 10001)
```

```
n4 = int(len(dataf)/fs * 25000)
```

توضیح

در این قسمت محدوده‌های نویزی مشخص می‌شوند.

تقریباً:

- زیر ۹۰ هرتز → نویزهای فرکانس پایین
- بالاتر از ۱۰۰۰۱ هرتز → نویزهای فرکانس بالا

12) کاهش نویز

```
dataf[n3:n4] /= 60
```

و

```
dataf[n1:n2] /= 60
```

توضیح

دامنه این فرکانس‌ها ۶۰ برابر کوچکتر می‌شود.

در نتیجه نویزها تضعیف می‌شوند.

همین کار برای قسمت متقارن طیف نیز انجام شده است.

(13) تقویت هارمونیک‌ها

```
dataf[k1:k2] *= 10
```

توضیح

فرکانس‌های بین

1000 Hz

تا

10000 Hz

ده برابر تقویت می‌شوند.

این محدوده بیشترین اطلاعات گفتار انسان را دارد؛ بنابراین باعث واضح‌تر شدن صدا می‌شود.

(14) رسم طیف جدید

```
plt.figure(3)
```

...

توضیح

بعد از اعمال تغییرات، دوباره طیف فرکانسی رسم می‌شود تا تفاوت قبل و بعد از پردازش قابل مشاهده باشد.

(15) انجام IFFT

```
enhanced = np.fft.ifft(dataf)
```

توضیح

پس از پایان پردازش در حوزه فرکانس، با استفاده از تبدیل فوریه معکوس (IFFT) سیگنال دوباره به حوزه زمان بازگردانده می‌شود.

(16) استخراج بخش حقیقی

```
clr = np.real(enhanced)
```

توضیح

خروجی IFFT ممکن است دارای قسمت موهومی بسیار کوچکی باشد. از آنجا که سیگنال صوتی واقعی فقط بخش حقیقی دارد، قسمت حقیقی استخراج می‌شود.

(17) تبدیل به داده ۱۶ بیتی

```
audio = clr.astype(np.int16)
```

توضیح

فایل‌های WAV معمولاً از داده‌های ۱۶ بیتی استفاده می‌کنند. بنابراین داده‌ها به نوع int16 تبدیل می‌شوند تا قابل ذخیره باشند.

(18) رسم سیگنال بهبود یافته

```
plt.figure(4)
```

```
plt.plot(t, clr)
```

توضیح

در این قسمت شکل موج صدای پردازش‌شده در حوزه زمان نمایش داده می‌شود تا با سیگنال اولیه مقایسه شود.

(19) نمایش نمودارها

```
plt.show()
```

توضیح

تمام نمودارهای رسم‌شده روی صفحه نمایش داده می‌شوند.

(20) ذخیره فایل صوتی

```
wavfile.write('improved.wav', fs, audio)
```

توضیح

در پایان، صدای پردازش شده با همان نرخ نمونه برداری اولیه (fs) در فایل جدیدی با نام **improved.wav** ذخیره می شود. این فایل نسخه بهبودیافته صدای اصلی است.